OpenDhi Group

AI Execution Layer

End-to-End Architecture

The governed orchestration layer that sits on top of every system we run — our own products and our customers' SAP, Oracle, Salesforce and more.

WHO THIS DOCUMENT IS FOR

Business team — read the Executive Summary and Part A. Plain language, no jargon.

Technical team — read Parts B, C, D and E. Stack internals, data flow, connector specifics, build state.

Everyone — the Executive Summary plus the companion interactive explainer is enough to onboard.

Version	1.0 — Architecture closure draft
Status	For business + technical review
Prepared for	Veera & Pradeep (technical) → Sudhir Kadam → Shrini (final sign-off)
Date	30 June 2026
Classification	Internal — Confidential

Contents

1. Executive Summary

THE ONE-LINE VERSION

We build one governed AI layer that sits **on top of** the systems we already run — our products and our customers' SAP, Oracle and Salesforce — so a single set of AI agents can safely run real tasks across all of them.

Today our systems are islands. ADT, Bhaiyaa, Wrapper+ and Newforce each hold their own data; a customer's SAP or Salesforce holds theirs. No single system sees a whole process end to end, so when something is stuck a person has to log into four systems to chase it. We cannot merge these systems, and ripping them out costs years.

So we don't replace anything — we sit on top. This is the same move Celonis and Palantir make: leave the systems of record where they are, and add the layers they are missing. We join their events into one journey, let an AI act on that journey, do it safely with rules and approvals, and surface it where people already work.

What that layer is made of

Six layers, read from the bottom up. Connectors plug into each system. Data Foundation keeps a shared set of IDs. The Context & Journey layer stitches events from every system into one timeline — this is the heart. Agents act on that timeline. Governance wraps every action in rules, approvals and an audit trail. Experience puts it inside the apps people already use.

Where we are today — the honest picture

[BUILT] ADT runs in production. The engineering team has built a read-only conversational analytics tool (ask-a-question, get-an-answer over one database).

[REUSABLE] Roughly 40% of that work is reusable plumbing we keep: authentication, the governance boundary, audit and observability.

[ROADMAP] The heart is still to build: the Journey/Context layer, cross-system event stitching, the agents that take action (write-back), and the SAP/Oracle/Salesforce connectors.

A reporting tool answers questions. **An execution layer** takes action. That gap — read-only to write-back — is exactly what the architecture in this document defines and what the next build phase delivers.

What we build first

The whole stack, thin, over ADT only, for one journey: the monthly payroll run. Prove the full path end to end on one real flow, then add more journeys, then connect customer systems like SAP and Salesforce.

Part A — The Business View

For the business team. *Plain language, the problem, the idea, and what it does. No technical detail is needed to follow this part.*

2. The Problem We Are Solving

Start with Babu. Babu is a construction worker. He clocks in on ADT. His attendance lives in ADT. His pay is calculated somewhere else. His wage card is issued by RBL. If his employer also runs SAP, the cost of his labour has to land in SAP too. Five systems, one person, one simple question: did Babu get paid correctly and on time?

Right now, nobody can answer that in one place. Each system knows its own slice. When a payment is stuck, a human logs into ADT to check attendance, into the payroll system to check the calculation, into the bank portal to check the transfer, and into SAP to check the expense posted. Four logins, four screens, one stuck payment.

> **THE PAIN, IN ONE LINE**
> Our systems and our customers' systems are separate islands. No one system sees the whole process, so people become the glue — logging into many systems by hand to move one thing forward.

3. The Insight

You cannot merge SAP and Salesforce, and replacing them takes years. So instead of replacing the islands, we build a bridge that sits above all of them. The islands stay exactly where they are. The bridge sees across all of them and can act.

This is a proven category, not a new bet. Celonis and Palantir built billion-dollar businesses doing exactly this: sitting on top of the systems a company already runs and adding the missing layers. Our edge is not being first — it is being lighter, sovereign (data stays in the customer's control), and deep in specific verticals like workforce and hyperlocal commerce.

> **THE LINE TO SAY OUT LOUD**
> *"We sit on top of the systems you already run — SAP, Salesforce, ADT — and add the layers they're missing: we join their events into one journey, let an AI act on it, safely, where people already work. We build it thin over the payroll run first, then widen it."*

4. What It Is — One Idea, Six Abilities

Hold one idea: a single, brilliant assistant for the whole company. Not one assistant bolted onto each product — one assistant that can see and act across all of them. That assistant has six abilities. Each ability is one layer of the architecture.

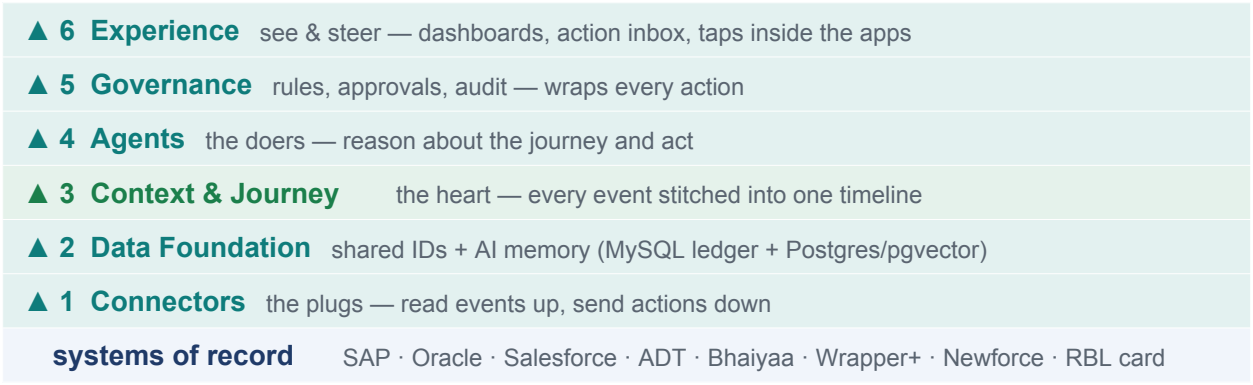
Ability (plain)	Layer name (technical)	What it means
1. Plug in	Connectors	Reach into each system to read what's happening and send back actions.
2. Remember	Data Foundation	Keep one shared set of "things" and IDs (an Employee, an Order) so every system's data can be linked.

3. Follow the journey **the heart**	Context & Journey	Stitch events from every system into one timeline, so we see the whole process — not four disconnected pieces.
4. Act	Agents	An AI reasons about the journey and does the next step — load a card, post an expense, raise a ticket.
5. Stay safe	Governance	Every action obeys rules, asks a human when it matters, and is written to an audit trail.
6. Show up where you work	Experience	Surface all of this inside the apps and dashboards people already use — not a new system to learn.

Why "follow the journey" is the heart: anyone can bolt a chatbot onto a single product. What makes this categorically different — not just more efficient — is stitching events across systems into one timeline. That is the thing per-product AI can never do, and it is the thing customers cannot easily build themselves.

5. Where It Sits

Picture a stack. At the bottom are the systems of record — the islands that already exist and stay exactly where they are. On top of them sits our layer. Data rises up the stack into the journey; actions flow back down into the systems.



Read it bottom to top: the layer assembles on top of what already exists. Context flows up the left rail; actions flow down the right. Nothing in the base is replaced.

6. What It Does — Walk One Real Task

Babu's payday, in plain language. One event travels up the stack and one action travels back down.

1. **Babu clocks in on ADT.** That single event is picked up by the **Connector**.
2. It's stored against Babu's shared ID in the **Data Foundation** — now it can be linked to everything else about him.

3. The **Context & Journey** layer stitches it into Babu's timeline: days worked, rate, prior payments — one continuous story.
4. At month end, an **Agent** reads that timeline, computes Babu's pay, and proposes the action: load his RBL card.
5. **Governance** checks the rule ("disbursals over a threshold need a human OK"), routes it for approval, and logs everything.
6. A manager sees it in the **Experience** layer's action inbox, taps approve. The card is loaded. If the employer runs SAP, the cost posts back to SAP as a reconciled expense line — automatically.

> **THE POINT**
> No one logged into four systems. The journey was visible in one place, the AI did the work, a human approved the one step that mattered, and every system stayed updated. That is the whole product in one event.

7. How We Are Different

Celonis and Palantir own this category broadly. We are not claiming to out-build them at their own game. We are claiming a specific lane.

Our edge	What it means in practice
Lighter	Faster to stand up and cheaper to run than a full Palantir/Celonis deployment — built thin, one journey at a time.
Sovereign	Data stays in the customer's control. The layer reads live and copies as little as possible; the system of record stays the source of truth.
Vertical-deep	We go deep in workforce (ADT), hyperlocal commerce (Bhaiyaa) and SAP extensibility (Wrapper+) — not shallow-everywhere.

8. Build State at a Glance

We are honest about what exists today. This table is the same one the technical team uses — it just hides the detail. The architecture below is the target; the tags say where we are now.

Layer	Status today	Tag
Connectors	SAP/Oracle/Salesforce connectors not yet built. Internal product reads exist in pieces.	ROADMAP
Data Foundation	MySQL is live. The Postgres + pgvector AI side and shared master-data model are to build.	PART / ROADMAP
Context & Journey	Not built. This is the heart and the main new work.	ROADMAP
Agents	A read-only analytics tool exists; the acting agent loop (write-back) is to build.	PART / ROADMAP
Governance	The boundary, audit and access checks largely exist as reusable plumbing.	REUSABLE

Experience	ADT; the action inbox and embedded approvals are to build.	PART / ROADMAP

Part B — The Technical View

For the technical team. *Stack internals, data flow, the data foundation, the agent loop, governance, and the reliability model. Build-state tags are inline throughout.*

9. The Six-Layer Stack, in Detail

Each layer has one job and a clear boundary. The principle that holds the whole thing together: the model proposes, a controlled program executes. The LLM never touches a database or an external system directly — it returns a structured intent, and deterministic code carries it out through a tool, behind governance.

Layer 1 — Connectors

Job: one adapter per system that reads events/data and sends actions. A connector is our plug on our side; the customer's integration (SAP CPI, Oracle OIC) is their plug on their side. They meet in the middle over APIs.

- Reads live and copies the minimum needed — it is not ETL and does not bulk-replicate the system of record.
- Two-way by design: events flow up, approved actions flow back down (write-back).
- Transport: REST/OData first; SOAP, SFTP/file, and message queues where a system requires it. **[ROADMAP]**

Layer 2 — Data Foundation

Job: keep a shared set of business objects and IDs so events from different systems can be linked — without merging the underlying databases.

- **MySQL** remains the transactional engine / ledger of live working records. **[BUILT]**
- **PostgreSQL + pgvector** is the AI/RAG side: embeddings, semantic search, and the memory the agents reason over (per Sudhir Kadam's recommendation). **[ROADMAP]**
- Master data is **linked, not merged**: we hold shared IDs (Employee, Order, Invoice) mapped across systems and fetch the live record per request. Each product keeps its own database. **[ROADMAP]**

Layer 3 — Context & Journey the heart

Job: stitch every relevant event into one ordered timeline keyed by a **Journey ID**. A journey is a real process — a payroll run, an expense claim, an order — not a single product's log.

- Cross-system event stitching: an ADT clock-in, a card load, and an SAP posting all attach to the same journey.

- This is what makes the layer categorically different from per-product AI. Without it, you have four chatbots; with it, you have one assistant that understands a whole process.
- Stored as events + journeys tables (MySQL) with vector context (pgvector) for retrieval. **[ROADMAP]**

Layer 4 — Agents

Job: reason about a journey and take the next step. A standard LLM, exposed to tools via **MCP** (Model Context Protocol). One agent is configured per journey; an orchestrator routes work between them.

- **Model proposes, program executes**: the LLM returns a structured tool call (e.g. `disburse(employee_id, amount)`); deterministic code validates and runs it.
- Tools are explicit and typed — `compute_pay()`, `disburse()`, `post_expense()`. The agent cannot reach a database directly. **[ROADMAP]**
- Today's read-only analytics tool is the seed of this layer (it already does "model proposes a query, program runs it") but has no write-back and no journey context. **[REUSABLE]**

Layer 5 — Governance

Job: wrap every action in rules, approvals and an audit trail. This is the layer that makes autonomy safe enough to ship.

- RBAC + policy engine: who may trigger what, and under which conditions. **[REUSABLE]**
- No direct DB writes — every change goes through an API behind this boundary.
- Full prompt/response audit: every model input and output is logged.
- Human-in-the-loop for critical actions: disbursals, payouts, refunds, bulk changes, seller blocking.
- **Best-effort compensating actions with an audit trail**: if a multi-step action partly fails, we reverse what we can and flag the rest for manual reconciliation. Some steps cannot be cleanly undone — we are honest about that, and the audit trail is the safety net.

Layer 6 — Experience

Job: surface everything where people already work — embedded in ADT, Bhaiyaa, or a chat tool — not as a separate system to learn.

- Action inbox: pending approvals with full journey context for a one-tap decision.
- Dashboards over the journey timeline; conversational query for ad-hoc questions.
- Embedded in existing product UIs via the same APIs. **[ROADMAP]**

10. Data Flow — Up and Down

Two directions, one rule. Context flows up the stack; actions flow down. The Journey ID is the thread that ties a rising event to a descending action.

Direction	Up the stack (sensing)	Down the stack (acting)
Trigger	An event happens in a system (clock-in, transaction, status change).	An agent decides on the next step and governance approves it.

Path	System → Connector → Data Foundation → Context/Journey → Agent	Agent → Governance (rules + approval) → Connector → System
What moves	The event, attached to a Journey ID, becomes part of the timeline.	A typed, validated action (a tool call) is executed and written back.
Result	The whole process is visible in one place.	The system of record is updated; the audit trail records it.

11. Data Foundation — Where Data Lives

THE PRINCIPLE: LINKED, NOT MERGED

We do not build one giant shared database. Each product and each customer system keeps its own data. We hold shared IDs that map the same business object across systems, and we fetch the live record per request. This keeps the system of record authoritative and keeps us sovereign.

Store	What it holds	Role
MySQL	Live transactional records; events and journeys tables.	Transactional engine / ledger. **[BUILT]**
PostgreSQL + pgvector	Embeddings, semantic indexes, retrieval context for agents.	AI / RAG memory side. **[ROADMAP]**
Master-data map	Shared IDs: Employee, Order, Invoice — mapped across systems.	The linking table, not a copy of the data. **[ROADMAP]**

Why two databases: MySQL is excellent for the transactional ledger we already run; PostgreSQL with pgvector adds first-class vector search for the AI side. We keep MySQL rather than migrate, and add Postgres alongside it — the recommendation we validated with Sudhir Kadam.

12. The Context & Journey Layer — Why It Is the Heart

This is the single most important idea in the architecture. Everything else is plumbing around it.

Per-product AI can answer questions about one system. It can never see a process that spans systems, because no single system holds the whole process. The Context layer constructs that whole: it stitches an ADT event, a bank transaction, and an SAP posting onto one Journey ID, in order, as one timeline.

THE TEST FOR "CATEGORICALLY DIFFERENT"

If a competitor bolts a chatbot onto each product, they get faster lookups. They do not get cross-system journeys. The Context layer is the thing that is hard to spec, hard to copy, and the reason the layer is a category, not a feature.

13. Agents — How Action Happens

The governing principle, restated: *the model proposes, a controlled program executes.* The LLM is a reasoning engine that emits structured intent. It does not run SQL, call bank APIs, or write to SAP. Deterministic, reviewed code does that — through typed tools, behind governance.

1. An agent is configured per journey (payroll agent, expense agent, order agent). It is given only the tools that journey needs.
2. It reads the journey timeline (context) and decides the next step.
3. It emits a typed tool call. Example: `disburse(employee_id=BABU01, amount=12450, currency=INR)`.
4. The orchestrator routes multi-step or cross-journey work and handles retries.
5. Every call passes through governance before it executes. Nothing autonomous bypasses the rules.

[REUSABLE] *The existing read-only analytics tool already embodies step 3 for queries (natural language → validated query → execute). Extending it to actions means adding write tools, the journey context it reads from, and the MCP acting loop.*

14. Governance — The Safety Wrap

Governance is not a feature bolted on at the end; it is the boundary every action crosses. The technical team has already built much of this as reusable plumbing.

Control	What it does	Status
RBAC + policy engine	Decides who may trigger which action, under which conditions.	REUSABLE
No direct DB writes	All changes go through APIs behind the governance boundary.	REUSABLE
Full prompt/response audit	Every model input and output is logged and queryable.	REUSABLE
Human-in-the-loop	Critical actions (disbursals, payouts, refunds, bulk changes) require explicit approval.	PART / ROADMAP
Compensating actions	Partial failures: reverse what we can, flag the rest for manual reconciliation, audit all of it.	ROADMAP

On compensating transactions, the honest framing: “best-effort compensating actions with an audit trail.” Some steps can be reversed automatically; some can only be flagged for a human. We do not claim clean two-phase rollback across external systems we do not control. This framing is also the stronger one for a “hard to spec, hard to copy” posture.

15. Experience — Where People Work

The layer is invisible until a human is needed. When a decision matters, it appears as a one-tap approval inside the tool the person already uses, carrying the full journey context so the decision is informed, not blind.

- Action inbox with journey context — approve/reject in one place.
- Timeline dashboards and conversational query for ad-hoc questions.
- Embedded in ADT, Bhaiyaa and chat tools via the same governed APIs — no new system to adopt.

16. Reliability & the Centralization Question

THE OBJECTION (state it plainly)
A shared layer on top of everything is a single point of failure and a potential bottleneck. If it goes down, does everything stop?

The answer: no. The design addresses this directly.

- **Stateless and horizontally scalable** — the layer holds no session state of its own; add instances behind a load balancer to scale.
- **Products keep working without it** — ADT, Bhaiyaa and the rest continue to function as normal apps if the layer is down. It augments; it is not in the critical path of the base products.
- **Centralized failure is cheaper to fix once** — one place to patch a bug beats the same bug fixed four times across four product teams. Centralization is a feature here, not only a risk.

Part C — Third-Party Systems (Deep)

For the technical team and pre-sales. *How SAP, Oracle, Salesforce and other systems connect — APIs, middleware, clean-core, licensing risk and write-back specifics.*

17. The Connector Pattern (General)

One pattern, repeated per system. We never move into the customer's system. We sit beside it and connect over its supported integration surface.

THE TWO-PLUG RULE
Our connector is the plug on OUR side. The customer's integration tool (SAP CPI, Oracle OIC, a Salesforce API) is the plug on THEIR side. They meet in the middle over APIs. The customer's system of record stays the source of truth; we read the live slice we need and write approved actions back.

Property	How it works
Read	Live, on demand. We pull the minimum active slice per request. We do not bulk-replicate the system of record (not ETL).

Write-back	Two-way. Approved actions are pushed back as native records (an expense line, a status update) via the system's write APIs.
Transport	REST/OData first; SOAP, SFTP/file and message queues where a system mandates them.
Identity	Each external record maps to a shared ID in our master-data map, so it joins the right journey.
Boundary	Every read and write crosses the governance layer; nothing the agent does bypasses rules and audit.

18. SAP — Deep Dive

Get one thing right first: SAP S/4HANA, SAP CPI and our layer are three separate things. Conflating them is the most common mistake in customer conversations.

Component	What it is
SAP S/4HANA	The vault — the customer's ERP and system of record. Holds the data and the financial truth. Stays exactly where it is; we sit beside it, never inside it.
SAP CPI	Cloud Integration, part of SAP Integration Suite on SAP BTP. The courier and translator — runs separately, holds no data, just carries and converts messages between S/4HANA and other systems.
Edge Integration Cell	The on-prem / hybrid runtime of Integration Suite (a Kubernetes runtime, GA March 2024). The strategic on-prem successor to PI/PO — used where data sovereignty requires integration to run inside the customer's own estate.
Our AI Execution Layer	The intelligence. Sits beside the SAP estate. SAP data enters and exits through our Connector, which talks to CPI (cloud) or Edge Integration Cell (on-prem). Their SAP data then rides the exact same six-layer stack our ADT data does.

Where SAP sits in the stack

SAP is a system of record at the base. CPI / Edge Integration Cell is the on-ramp at the Connector layer — a different on-ramp at the floor, nothing more. We stand on our own Data Foundation, never on CPI. For an L&T or an Ashida, their SAP events rise through Context → Agents → Governance → Experience identically to every other source.

Clean-core and SAP BTP

SAP's clean-core principle (August 2025 Level A definition) requires side-by-side extensions to run on SAP BTP rather than as modifications inside the S/4HANA core. **Implication for us:** we position as a side-by-side AI execution/experience layer that coexists with SAP-native integration — not a like-for-like middleware replacement, and not a core modification. This keeps the customer clean-core compliant and keeps us out of the ERP's upgrade path.

Risks to name explicitly in customer materials

Pre-sales must surface these proactively. Each is verifiable against an SAP source; do not soften them.

Risk / fact	Why it matters	Source
SAP Digital Access document licensing	Indirect/digital access is licensed per document created (nine document types). This is a real and often-missed TCO line that undercuts any “predictable TCO” claim.	SAP Note 2942344
PI/PO end of mainstream maintenance	Process Integration / Process Orchestration reaches end of mainstream maintenance 31 December 2027. Customers on PI/PO must plan a move (Integration Suite / Edge Integration Cell).	SAP KBA 3463462
SAP API Policy v.4.2026a	Restricts autonomous AI-driven API sequencing; enforcement reported from 9 June 2026. Our “model proposes, program executes” pattern is designed to stay compliant — but it must be designed in, not assumed.	SAP API Policy v.4.2026a
HANA OS support	SAP HANA supports only SUSE and Red Hat enterprise Linux — not plain Ubuntu. Any “Ubuntu-for-SAP” claim in a customer’s stack is a credibility risk.	SAP Note 2235581

POSITIONING GUARDRAIL

We coexist with SAP-native integration; we do not replace it. For on-prem/sovereign deployments the SAP-correct path is Edge Integration Cell, and our layer sits above it. Leading with “we replace your middleware” in front of a conservative SAP practice loses credibility.

SAP write-back — the specifics

Two flows make the integration two-way and concrete:

- **Approved claim → card load:** when an expense claim is approved in the journey, the agent triggers a card load on the RBL/Paycraft side.
- **Card transaction → reconciled expense line:** when the card is spent, the transaction is pushed back into SAP as an auto-reconciled expense line via CPI / Edge Integration Cell.
- **Failure handling:** best-effort compensating action with an audit trail — reverse the card load where possible, otherwise flag the SAP posting for manual reconciliation.

19. Oracle — Deep Dive

Same pattern, Oracle vocabulary. Oracle ERP / Fusion is the system of record; **Oracle Integration Cloud (OIC)** is the customer-side integration plug, the Oracle equivalent of SAP CPI. Our Connector meets OIC over APIs.

Component	Role
Oracle ERP / Fusion Cloud	System of record at the base — payroll, financials, procurement. Stays where it is.
Oracle Integration Cloud (OIC)	The customer-side middleware/iPaaS — the Oracle counterpart to SAP CPI. Carries and translates; holds no master state for us.

REST / SOAP adapters	Oracle exposes REST and SOAP web services and a large adapter catalogue; our Connector consumes these for read and write-back.
Our Connector	Reads the live slice, writes approved actions back (e.g. a payable, a journal line). Same governance boundary as every other source.

Write-back example: an approved disbursal posts back to Oracle as a payables/expense entry; the same best-effort compensating-action discipline applies on partial failure.

20. Salesforce & Other Third-Party Systems

CRM and the long tail follow the same rule. Salesforce is the system of record for CRM; the Connector talks to it over its REST and Bulk APIs. Anything else — a legacy app, a flat-file feed, a message bus — connects through generic REST/SOAP/SFTP adapters.

System	How it connects
Salesforce	Connector over Salesforce REST / Bulk / Streaming APIs. Reads accounts, opportunities, cases; writes back updates and tasks. (Relevant to the Racha.ai / Samba TV “Salesforce replacement” narrative — we sit beside it first, displace later.)
Microsoft / other ERP	REST/OData where available; otherwise SOAP or file-based exchange.
RBL Bank / Paycraft card	A system of record at the base for card issuance and spend. Our layer is the card and spend layer plugging into RBL’s EMS via APIs — card loads on approval, transactions pushed back as reconciled expense lines.
Generic / legacy	REST, SOAP, SFTP/file, or message-queue adapters. The connector pattern does not change — only the transport does.

> **ONE MORE BOUNDARY NOTE (RBL / EMS)**
> With RBL we are the card and spend layer plugging into RBL’s already-built EMS via APIs — not EMS-to-EMS. We contribute the multi-wallet card, instant disbursal, real-time spend controls and ADT employee identity. Two-way: approved claims trigger card loads; card transactions push back as auto-reconciled expense lines.

21. Write-Back & Two-Way Integration — the Specifics

The write-back path is what separates an execution layer from a reporting tool. It is also the part most likely to be challenged in technical review, so it is specified plainly here.

Step	Trigger	Result in the system of record
Disburse on approval	Expense claim or payroll run approved in the journey.	Card load executed (RBL/Paycraft); funds available to the worker.
Post the cost	Card transaction settles.	Auto-reconciled expense line written to SAP/Oracle via CPI / OIC.
Reconcile	Transaction matched to the originating claim by Journey ID.	Expense line linked to claim; books stay consistent.

Handle failure	A step partly fails (e.g. card loads, SAP posting rejects).	Best-effort reversal where possible; otherwise flagged for manual reconciliation, with full audit trail.

Part D — End-to-End Worked Example

The payroll journey, fully traced. *First the all-internal path (ADT + RBL), then the SAP variant showing exactly where SAP enters and exits.*

22. The Payroll Journey, Step by Step

Path A — internal only (ADT + RBL card)

#	Layer	What happens
1	Connectors	Babu clocks in on ADT. The ADT connector captures the attendance event.
2	Data Foundation	The event is stored against Babu's shared Employee ID; MySQL ledger updated.
3	Context & Journey	The event is stitched onto Babu's payroll Journey ID: days worked, rate, prior pay — one timeline.
4	Agents	At month end, the payroll agent reads the timeline, runs compute_pay(), and proposes disburse(BABU01, 12450).
5	Governance	Rule check: disbursal over threshold → needs human approval. Routed to the manager; everything logged.
6	Experience	Manager sees it in the action inbox with full context, taps approve.
7	Connectors (down)	Approved action flows down: RBL/Paycraft card load executed. Babu is paid.

Path B — SAP variant (employer runs S/4HANA)

Everything in Path A is identical. SAP enters at two points only:

#	Where SAP enters	What happens
4a	Read (optional)	If cost-centre or rate data lives in SAP, the SAP connector reads it live via CPI / Edge Integration Cell and adds it to the journey — it does not copy SAP wholesale.
7a	Write-back	After the card load, the labour cost posts back to S/4HANA as a reconciled expense/journal line via CPI / Edge Integration Cell.
7b	Reconcile	The card transaction, when it settles, is matched to this journey and confirmed against the SAP posting.

WHAT THIS PROVES

The same stack handles an internal source (ADT) and an external system of record (SAP) with no change to the journey, the agent, or the governance. SAP is just another on-ramp at the Connector floor and another write target on the way down. That is the whole architecture, working.

Part E — Build State & Sequence

The honest engineering picture. *What exists, what is reusable, what is missing, and the order we build in.*

23. What Is Built Today

- [BUILT] ADT Needs to be built: Client → Cache → Application Layer (Consumer/Seller services) → MySQL/RDS → Edge, on AWS. This is the only fully-built product architecture.
- [BUILT] A read-only conversational analytics tool (natural language → validated query → execute, over a single database). It answers questions; it does not act.

THE KEY DISTINCTION

The analytics tool is a reporting engine, not an execution layer. On our own framing it sits in the ETL/Analytics corner we explicitly excluded. It is genuinely useful and a real head-start — but it is not the layer this document describes. Naming that gap clearly is what keeps everyone aligned.

24. What Is Reusable, What Is Missing

Reusable plumbing (~40%) — keep	Missing — the core build
Authentication	Context / Journey layer (cross-system stitching)
Governance boundary	Write-back actions (the acting agent loop)
Audit	MCP tools + orchestrator
Observability	Postgres + pgvector AI side; shared master-data map
Query-validation discipline (model proposes, program runs)	SAP / Oracle / Salesforce connectors + write-back

25. The Build Sequence

Thin and vertical, not broad and shallow. Prove the whole path on one journey before widening.

1. **Phase 1 —** the full six-layer stack, thin, over ADT only, for the monthly payroll journey. End-to-end: clock-in to card load to (optional) SAP posting.
2. **Phase 2 —** add more journeys on ADT (expense claims, advances), reusing the same stack.

3. **Phase 3 —** connect customer systems of record: SAP (CPI / Edge Integration Cell), then Salesforce/Oracle as deals require.
4. **Phase 4 —** widen across products (Bhaiyaa, Wrapper+, Racha.ai) once the pattern is proven and the connectors are hardened.

26. Open Questions to Validate

These are the honest open items to close with the technical reviewers before final sign-off. They are listed so they are not glossed over.

Open question	Owner / how we close it
Can the vector store become real master data, or does it stay retrieval-only alongside the master-data map?	Veera / Pradeep — data-model decision.
Sovereignty / deployment model per customer (cloud vs on-prem Edge Integration Cell).	Per-deal; Veera to set the default posture.
SAP API Policy v.4.2026a compliance for the acting agent loop.	Design review — confirm “model proposes, program executes” satisfies the policy.
Exact write-back contracts (SAP/Oracle document types, idempotency, reconciliation keys).	Connector design with Tarak / Shrikant.
Resourcing and costing for Phase 1 (team, timeline, infra).	Plan alongside backlog upgrades; Sada → Sudhir Kadam → Shrini.

APPROVAL PATH

Sada + Pradeep align first → brief and align with Veera → all three aligned → Sada takes it to Sudhir Kadam → Shrini for final sign-off. **No external or investor version goes out until the technical team has validated this document.**

Part F — Reference

27. Glossary

Term	Plain meaning
AI Execution Layer	Our governed layer that sits on top of all systems so a single set of AI agents can act across them. The subject of this document.
System of record	The authoritative source for a piece of data (SAP for finance, ADT for attendance). Stays where it is; we sit beside it.
Connector	Our adapter that reads from and writes to one external system. Our plug on our side.
SAP CPI	Cloud Integration on SAP BTP — SAP’s courier/translator middleware. Holds no data; carries messages. The customer’s plug on their side.

Edge Integration Cell	The on-prem/hybrid runtime of SAP Integration Suite (GA March 2024). The on-prem successor to PI/PO, for sovereign deployments.
SAP BTP	Business Technology Platform — where SAP-compliant side-by-side extensions run (clean-core).
Clean-core	SAP's principle of keeping the S/4HANA core unmodified; extensions live side-by-side on BTP. Level A definition, August 2025.
Oracle OIC	Oracle Integration Cloud — Oracle's equivalent of SAP CPI; the customer-side integration plug for Oracle estates.
Context / Journey layer	The heart of the architecture: stitches events from all systems into one ordered timeline, keyed by a Journey ID.
Journey ID	The identifier that ties every event and action in one real process (a payroll run, a claim) into a single timeline.
Master data (linked, not merged)	A map of shared IDs across systems. We link the same object across systems without copying the databases into one.
MCP	Model Context Protocol — how the LLM is given access to typed tools to act, behind governance.
Model proposes, program executes	The core safety principle: the LLM returns structured intent; deterministic code performs the action.
Write-back	Pushing an approved action back into the system of record (a card load, an expense line). What makes the layer an execution layer, not a report.
Compensating action	On partial failure, reversing what can be reversed and flagging the rest for manual reconciliation — best-effort, fully audited.
RBAC	Role-Based Access Control — who is allowed to trigger which action.
pgvector	A PostgreSQL extension for vector/semantic search, used on the AI/RAG side of the Data Foundation.

28. How to Read This Document, by Role

You are…	Read
Business / commercial	Executive Summary + Part A. The companion interactive explainer covers the same ground one idea at a time.
Engineering / architecture	Parts B, C, D and E. Pay closest attention to the Context layer (12), the agent loop (13), governance (14), the SAP deep dive (18) and build state (23–26).
Pre-sales	Part A for the narrative, Part C for the SAP/Oracle/Salesforce specifics and the risks to name in customer materials.
Leadership (Sudhir Kadam, Shrini)	Executive Summary + the build-state and open-questions sections (8, 23–26) + the approval path.

Companion artifact: *an interactive explainer (one idea per screen, depth behind a tap) accompanies this document for onboarding anyone — technical or not — in about ten minutes.*

A note on honesty: *every “built” claim in this document reflects what exists today; every “roadmap” claim is work the architecture defines and the next phase delivers. We never present a roadmap feature as built — internally or to a customer.*